CHAPTER 17

Testing Legacy Code

You won't always get to start from scratch.

There's a strong possibility that you'll need to bring your testing skills to bear on a codebase that already exists. A codebase that—gasp—may not have a complete or accurate test suite to guide you.

Not all that long ago, there were no Rails legacy projects. Now there are Rails codebases that are old enough to attend middle school and beyond. These codebases tend to be large, and almost by definition they aren't using the latest tools. All the advice in this book so far is well and good, but what if you're not starting your TDD experience with a new application? How does testing change when you're dealing with a legacy codebase?

Entire books have been written on working with legacy codebases. In this chapter you'll focus on techniques for getting legacy code under test. There are many other issues you'll need to deal with in a legacy codebase. Getting the legacy system up and running can be a chore, and there's a variety of techniques for safely adding features to or refactoring existing code. I'll discuss those topics here only as they intersect with testing. For a more detailed look at managing a legacy project, check out Michael Feathers's excellent *Working Effectively with Legacy Code [Fea04]*.

What's a Legacy?

The phrase "legacy code" is often used as a synonym for "bad code." While the two categories often overlap, assuming the legacy code is always awful isn't the most helpful way to think about a legacy.

What makes code "legacy" is not the quality, but the extent to which you have access to the context in which it was created and the reasons previous developers made the choices they made.